通过 GPIO 交换矩阵的外设输入

© 小白 · 原创整理

输入信号通过 IO MUX 从 GPIO 管脚中读取。IUX 必须设置相应的管脚为 GPIO 功能。这样 GPIO 管脚的输入信号进就可以进入 GPIO 交换矩阵然后通过 GPIO 交换矩阵进入选择的外设输入.

IO MUX IO（多路复用器） 是 “物理总闸”

任何一个 GPIO 都要链外设 ,必须先过 IO MUX 这一关 ,不把他配置成 GPIO 功能 ,信号根本进不来，代码写死也没有任何反应。

交换矩阵是“智能路由”

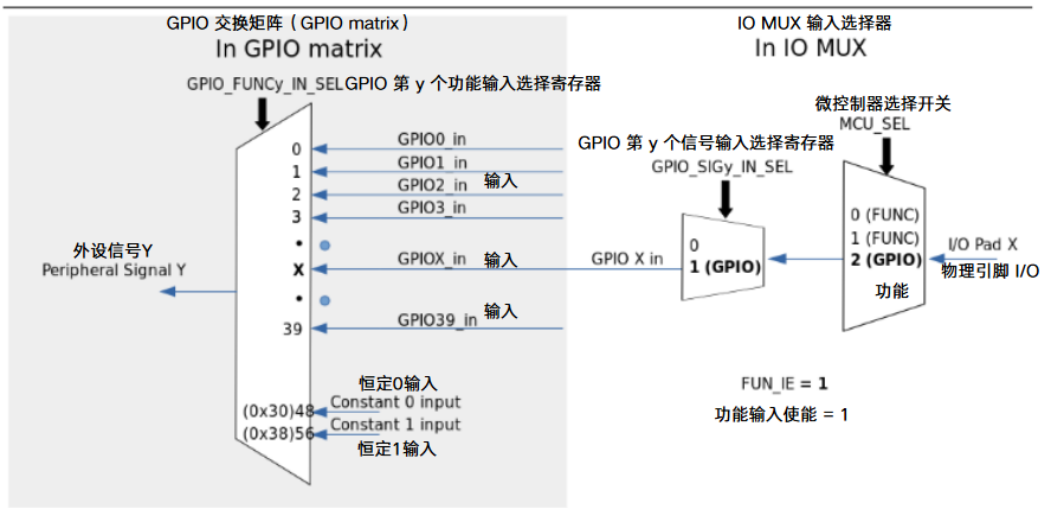
它把 34 个物理 GPIO，映射成 206 个逻辑信号索引。

比如想用 UART2 的 RX，它帮你把物理 GPIO16 路由送过去.

为什么不能乱接引脚？

配错索引，信号就丢了！

代码只是再操作寄存器，但硬件有物理限制！



告诉“交换矩阵”我要哪个引脚？

hw->GPIO_FUNCy_IN_SEL = X;

GPIO_SIGy_IN_SEL: 这个位是个**总开关**，设为 1，让外设走交换矩阵；

GPIO_FUNCy_IN_SEL：这个位是个**赋值口，**把 **X**(引脚号)

代码 GPIO_FUNC83_IN_SEL = 15；(RMT 信号去 83，去 GPIO 取数据)

把引脚 X 设置为“纯输入”

“禁止引脚输出！只允许输入！”

为什么要关输出？因为只读电平，不能输出电压，如果不关输出，引脚电平会乱跳，导致输出不准。

GPIO_FUNCx_ONE_SEL:设为 1，**开启输出控制**（我要管输出）

GPIO_ENABLE_DATA[x]:设为 0，**关闭输出**（关闭输出驱动器）

GPIO_FUNC15_OUT_SEL_CFG |= GPIO_FUNC15_OEN_SEL; // 开输出控制

GPIO_ENABLE_REG_REG &~(1 << 15); //关输出

打开引脚，输入信号！

MCU_SEL = 2 把引脚切换 GPIO 模式（不走外设专用路）

FUN_IE = 1 打开输入使能（硬件允许信号进来）

IO_MUX_GPIO15_REG = (2 << 0) | (1 << 13) //MCU_SEL = 2, FUN_IE = 1;

技巧！

GPIO_FUNCy_IN_SEL = 0x30 始终为 0；0x38 始终为 1

可以直接给外设送死电平，不接任何线

想测试某外设，但没硬件，或者想把某个输入强制设置为高电平

把 **GPIO_FUNCy_IN_SEL** 设置为 0x30.

外设永远读取到 0 设置 0x38 永远读 1.

硬件自动取反

置位 GPIO_FUNCy_IN_INV_SEL.

” 高电平变低，低电平变高，硬件自动来，不用代码写逻辑。”

比如按键是低电平触发但代码习惯高电平有效，硬件位换这个位，自动反转，代码逻辑不需要改！

1. 绑定：外设 Y 选引脚 X

hw_>GPIO_FUNCy_IN_SEL = X;

2. 输入；关输出（纯输入模式）

hw->GPIO_FUNCx_ONE_SEL = 1; // 开输出控制

hw->GPIO_ENABLE_DATA[x] = 0； // 关输出驱动器

3. 开通：IO MUX

hw->IO_MUX_x_REG = (2 << 0) | (1 << 13); MCU_SEL = 2,使能输入

实例内容：

1. 要把 RMT 第 0 通道的输出信号，物理接到 GPIO15 这个引脚上。

   “信号 83 以后就走 GPIO15”

2. 将 GPIO_FUNC83_IN_SEL_CFG_CFG 寄存器的 GPIO_CUNC83_IN_SEL 字段设置为 15.

   “信号索引号 83，负责在 GPIO15 这拿输入数据。”

   这是输入矩阵（IOMUX）最核心的一步：

   把外设信号 → 映射到具体引脚号。

3. 位置 GPIO_FUNC15_OUT_SEL_CFG_REG 中的 GPIO_FUNC15_ONE_SEL 位

   GPIO15 的“输出使能选择”位置写成 1。

   GPIO15 的输出方向，由外设（RMT）说的算

   别让软件覆盖外设信号。

   这一步是为了**防止软件误操作覆盖外设信号。**

4. 清零 GPIO_ENABLE_REG 的 bit 15.

   GPIO_ENABLE 寄存器是控制引脚方向的

   把 bit 清成 0，就相当于：

   GPIO = 输入模式

   因为 RMT 是输入外设，所以引脚必须是输入。

5. 配置 IO_MUX_GPIO15 的 MCU_SEL 为 2，置为 FUN_IE.

   GPIO 不在作为普通 GPIO 使用

   启动它的第二个复用功能（通常就是 RMT）

   FUN_IE = 1

   打开 GPIO15 的输入使能，让信号进入芯片。

   这两步是让**引脚功能真正生效。**

整体逻辑总结

这四步共同完成了一条物理链路：

RMT 外设信号→信号索引号 83→映射到 GPIO15→配置成输入功能→外设驱动生效

就是把 **RMT 信号物理接到 GPIO15** 的标准流程。

额外补充

ESP32 系列的 IO 映射一般分为两步：

1. **信号映射（IOMUX）**

   外设信号（比如 RMT、UART、I2C）=系统内部编号

   需要告诉系统："这个内部编号去哪个引脚。"

2. **引脚功能复用（IO MUX)**

   配置成外设功能，而不是普通 GPIO

© 小白 · 原创整理